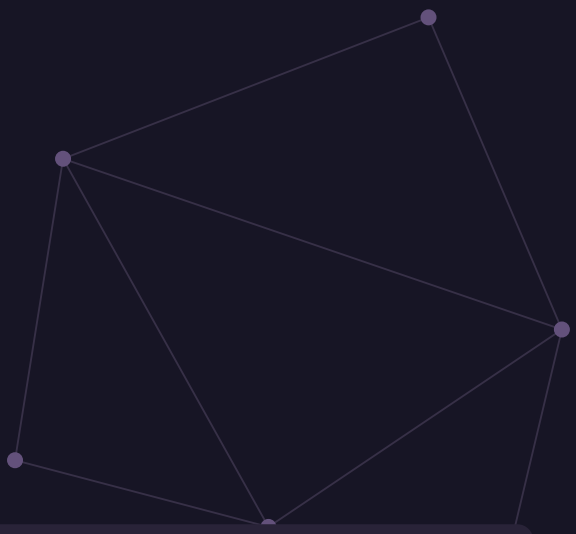
ORBITRON

ORBITRON / SECURITY RESEARCH

php2shell

Detection and response guide



Lab findings, log signatures and operational response steps.

CVE-2026-109618

CVE-2026-109619

9.2

CVSS v4.0

v1.2 / ENGLISH

21 October 2026

Prepared by the ORBITRON Team

00 Overview

CVE-2026-109618 and CVE-2026-109619 (php2shell; CVSS v4.0 9.2 each) are two PHP engine vulnerabilities involving memory exhaustion and invalid state retained across requests. They are fixed in **8.2.35 / 8.3.36 / 8.4.27 / 8.5.12**. This guide focuses on request-startup trigger conditions and the telemetry available to SOC teams. **A log match alone does not establish an attack attempt, an unpatched version or successful exploitation.** Evaluate request context, process telemetry and vendor package information together.

01 Scope and verification limits

This guide combines PHP's official advisories with ORBITRON's isolated lab results. The lab used unpatched/patched builds from official sources, upstream regression tests and additional controlled tests. No exploitation was triggered on production systems.

- **Official confirmation:** PHP released the four fixed versions. The relevant advisories are GHSA-w8hw-w9fx-j79w / CVE-2026-109618 and GHSA-6wmf-4rv5-p3rp / CVE-2026-109619. The source fix is titled Avoid state corruption via bailout.
- **Lab verification:** The 8.3.35/8.3.36 matrix below demonstrates differences in parsing and session error handling. A separate 8.4.26/8.4.27 test demonstrated controlled file writing during request startup using a preloaded class and prepared session data. Keep the versions and prerequisites of these two test series separate.
- **Evidence boundary:** A full memory-corruption → RCE chain (heap grooming, pointer hijack) was not developed. PHP's security advisories confirm RCE impact; the controlled file-write test alone is not evidence of remote RCE against an unprepared server.
- **Older versions:** PHP 8.1 and earlier are outside normal upstream support. However, the PHP security team supplied minimal source backports for 7.4, 8.0 and 8.1. These do not restart the normal support lifecycle. Verify with the distribution/LTS vendor that its package fixes both CVEs.
- **Scope:** PHP considers mod_php the highest-risk setup; other SAPIs serving web requests also require updating. FPM terminating a worker after failed startup prevents that process from carrying state into another request. The official CVE-2026-109619 advisory also describes an FPM path involving a suitable application operation after successful startup and an early consumer in a subsequent request.
- **Excluded generalisations:** Identical exploitation conditions across every historical PHP version and every CMS were not independently verified in this work.

02 How the bug works

For supported content types, PHP can automatically parse the request body before the application script runs. The paths examined here are:

1. **Memory consumption:** Numerous multipart headers, long/nested field names or data processed without a usable `Content-Length` can amplify the memory cost of an apparently small request. Some paths in the unpatched parser can bypass `post_max_size` enforcement. 2. **CVE-2026-109618:** When an allocation fails, an early exit through `zend_bailout()` can leave temporarily reachable internal data only partially initialised. Subsequent cleanup can use that invalid data. Application paths after startup can also cause

OOM. 3. **CVE-2026-109619:** Worker globals may retain references to request memory after it is freed. Early operations in a later request on the same worker may consume those references before normal initialisation resets them. Fatal errors other than OOM and some shutdown diagnostics can also leave such state; the two CVEs are not necessarily consecutive steps in a single chain.

The fix includes OOM handling deferred to safe points (`EG(delayed_oom)`), fallible allocation paths, parser checks and resets of relevant global state. Upload-progress deserialisation also uses **incomplete-classes-only** mode without class resolution. This does not mean PHP can no longer run out of memory or that every use of `longjmp` was removed.

The table below summarises only the lab's startup-OOM scenario; it is not a general immunity matrix.

SAPI	Startup-OOM result	Observed record	Reuse of the same worker
PHP-FPM	Worker terminates (`FPM_EXIT_SOFTWARE`)	`child <PID> exited with code 70` and an OOM error	That worker is not reused; other paths remain in scope
mod_php	Lab request returned 500; worker continued	OOM in the Apache error log	The same PID was reused; this alone does not prove UAF
CGI: process per request	Process terminates	Startup-OOM error	No subsequent request in that process; same-request risks are not excluded

`code 70` is not a SIGSEGV. Other startup errors can also produce this exit code; confirm OOM as the cause from the PHP error record.

03 Lab verification

Unpatched and patched builds were compiled with debug + AddressSanitizer; PHP's new regression tests were run on both.

Proof matrix (PHP's own tests; 8.3.35 unpatched, 8.3.36 patched):

Test	Unpatched	Patched	What it shows
Official regression tests (total)	5 of 27 fail	27/27 pass	Selected regression suite
Multipart `post_max_size` bypass (GHSA-2cv3)	FAIL - whole body ends up in `$_POST`	PASS - warning + data dropped	Controlled-OOM entry primitive
600,000 repeated part headers (GHSA-f883 007)	FAIL - unbounded accumulation → OOM	PASS - only 2 headers kept	Memory-exhaustion vector
Upload-progress, missing script (`rfc1867_no_user_code`)	FAIL - classes resolve, `$_SESSION` left behind	PASS - expected userland code does not run in this test	Pre-RINIT code execution
Invalid SID + upload-progress (`rfc1867_sid_invalid`)	FAIL - write attempted after read failed	PASS - nothing written	Source of the A4 signature (below)

Mitigation matrix - which interim setting actually stops the OOM (unpatched 8.3.35 php-cgi, with PHP's own test body):

Setting	Result
Default (memory_limit=128M)	OOM - `in Unknown on line 0`
`post_max_size=8M` alone	OOM (does not stop it)
`session.upload_progress.enabled=0` alone	OOM (does not stop it)
`session.use_strict_mode=1` alone	OOM (does not stop it)
`memory_limit=-1`	Completes (~218M peak)
Patched 8.3.36, default	Completes (~2M peak)

For this test body and configuration, `memory_limit=-1` was the only tested setting that avoided the PHP memory-limit abort. It is not a general protection guarantee: underlying allocation failures remain possible. Removing the PHP limit increases DoS risk; bound resource consumption with cgroup/container limits, request limits and concurrency controls. Reaching a cgroup limit may OOM-kill a worker/container; that is not the same mechanism as PHP's `memory_limit` error. Do not treat `RLIMIT_AS`/`RLIMIT_DATA` limits as equivalent. The figures are measurements from debug + ASan builds, not direct estimates of production memory requirements. ORBITRON also observed the same body exhausting the 128M limit in the distribution's 8.3.6 mod_php build.

04 Where the logs live

The paths below are common examples; they vary by distro, install method, container and vhost config. **Confirm the real path from the running service's configuration.**

Install / source	Records to collect	For what
Rocky / RHEL / Alma - Apache	`/var/log/httpd/error_log`, `/var/log/httpd/access_log`; vhost logs	PHP/Apache errors + related requests
Ubuntu / Debian - Apache	`/var/log/apache2/error.log`, `access.log`; vhost logs	Same; `ErrorLog`/`CustomLog` decide
nginx	`/var/log/nginx/error.log`, `access.log`	FastCGI/upstream errors, 5xx, request context
Rocky/RHEL - PHP-FPM	`/var/log/php-fpm/error.log` + pool PHP error file (e.g. `www-error.log`)	FPM process events + PHP fatal/warning
Ubuntu/Debian - PHP-FPM	`/var/log/php8.4-fpm.log` (name varies by version) + pool error file	Worker exits + PHP errors
systemd	`journalctl -u php\*-fpm\* -u httpd -u apache2 -u nginx`	Service errors, restarts, journal output
Container / Kubernetes	Container stdout/stderr + separate in-container log files	PHP/FPM/web logs with pod/container/node info
WAF / reverse proxy	Security events, rule matches, body-inspection errors	Multipart anomalies, blocking
Endpoint	EDR events; auditd `/var/log/audit/audit.log` if configured	Process/file/network activity from PHP/web workers

Important distinction: FPM's own `error_log` and PHP's `error_log` manage **different** records. Collecting only one misses the other. Collecting the nginx access log is not the same as collecting the PHP error log.

Finding the real path:

- **Apache:** `ErrorLog` / `CustomLog` in the active server and vhost config.
- **nginx:** `error_log` / `access_log` in the active `http`/`server`/`location` blocks.
- **FPM:** `error_log` in `php-fpm.conf`; `php_admin_value[error_log]`, `php_admin_flag[log_errors]`, `access.log` in pool files.
- **PHP:** the php.ini loaded by the SAPI that serves the request. Do **not** assume `php --ini` on the CLI matches the web SAPI.

Prerequisite: `log_errors=On` and `error_log` defined. On FPM, `catch_workers_output=yes` is recommended so worker stdout/stderr reaches the main log. Fatals are logged even with `display_errors=Off`.

05 Trigger signatures with log examples

The raw log lines below were produced by firing the real vector against an unpatched 8.3.x FPM and mod_php lab. These are **hunting signals** that require context; `...` marks abbreviated text.

A1 - Main trigger: OOM while parsing the request (CVE-2026-109618)

FPM (stderr forwarded to the main log):

LOG / PATTERN

```
WARNING: [pool www] child 849 said into stderr: "NOTICE: PHP message: PHP Fatal error:  Allowed memory
  size of 134217728 bytes exhausted ... in Unknown on line 0"
```

mod_php (Apache error_log):

LOG / PATTERN

```
[php:error] [pid 587] PHP Fatal error:  Allowed memory size of 134217728 bytes exhausted (tried to
  allocate 4096 bytes) in Unknown on line 0
```

Regex:

LOG / PATTERN

```
Allowed memory size of [0-9]+ bytes exhausted.* in Unknown on line 0
```

Interpretation: `in Unknown on line 0` means no script/line location is attached to the error; it does not uniquely identify multipart parsing or an attack. It was observed during startup OOM in the lab. Repeated requests reliably attributed to the same source, with increasing size or part count, strengthen suspicion.

A2 - Session ID warning during request startup

LOG / PATTERN

```
PHP Warning:  PHP Request Startup: Session ID is too long or contains illegal characters ... in Unknown
  on line 0
```

The `PHP Request Startup:` prefix identifies startup context. This session warning is consistent with the lab upload-progress path, but the prefix alone does not prove a particular handler or its ordering relative to RINIT. It can also occur on patched versions and malformed legitimate requests. Investigate it alongside unexpected multipart fields; do not assign HIGH/CRITICAL solely from this line.

A3 - Session read failure (same chain as A2)

LOG / PATTERN

```
PHP Warning:  PHP Request Startup: Failed to read session data: files (path: /var/lib/php/sessions) in
  Unknown on line 0
```

This does not distinguish patched from unpatched systems. Permissions, storage or session configuration failures can also cause it. Evaluate it with A2, multipart context and session-storage health.

A4 - Session write failure during request startup

LOG / PATTERN

```
PHP Warning:  PHP Request Startup: Failed to write session data (files) ... in Unknown on line 0
```

In the `rfc1867_sid_invalid` regression scenario, the unpatched version attempted a write after a failed read; the patched version did not. **That test difference does not make the generic "Failed to write session data" message exclusive to vulnerable versions.** Independent write/permission/storage failures can also occur on patched systems. A successful write produces no such error. A4 therefore proves neither version nor exploitation, and its absence does not establish safety. Verify patches using package information, the vendor advisory and the running process.

A5 - Content-Length and body-limit warnings

LOG / PATTERN

```
POST Content-Length of [0-9]+ bytes exceeds the limit of [0-9]+ bytes
Actual POST length does not match Content-Length, and exceeds [0-9]+ bytes
```

These are two different warning patterns. The first can result from an ordinary oversized POST rejected by a limit; it does not by itself show a false `Content-Length` or successful bypass. The second indicates a declared/actual length mismatch and limit exceedance. Verify how the proxy/WAF measures transferred and decoded body data; HTTP framing or proxy normalisation may reject a suspicious request before PHP receives it.

A6 - WAF / body inspection (a rule, not a log)

In the multipart body, if the app does not legitimately use them, look for:

- a `name="PHP_SESSION_UPLOAD_PROGRESS"` form field
- a `name="PHPSESSID"` SID supplied in the body
- megabytes of repeated `Content-Disposition` / `Content-Type` headers; hundreds of thousands of headers in one part; `Content-Length: 0` with a large body

Field names: `PHP_SESSION_UPLOAD_PROGRESS` and `PHPSESSID` are defaults; adapt them to the effective `session.upload_progress.name` and `session.name`. Presence alone does not show malicious intent.

06 Group B - Worker failures and investigation signals

These records help investigate service failures, resource exhaustion or possible memory corruption. **They cannot establish successful exploitation on their own.**

- **FPM** `exited with code 70`**:** Observed in the examined startup-OOM tests. It is not SIGSEGV; other startup errors can also produce it. Attribute it to OOM only with A1 and worker context.
- **SIGSEGV / SIGABRT:** May accompany memory corruption, but extension defects, assertions and other software faults can also produce them. A crash does not prove heap grooming or successful exploitation.

Where	What to look for
FPM master log	`child [0-9]+ exited with code 70` (OOM-startup) · `exited on signal 11 (SIGSEGV)` · `exited on signal 6 (SIGABRT)` · short-interval worker respawn clusters
Apache error log	`seg fault or similar nasty error detected in the child` · `child pid [0-9]+ exit signal Segmentation fault \(11\)`
nginx error log	`upstream prematurely closed connection` · `recv\(\) failed` + 502 bursts in the access log
Apache + FPM	`AH01071: Got error 'PHP message: ...'` (PHP message carried into the Apache log)
Access log (correlation)	multipart POST from the same IP → immediately 500/502/503 → the same IP continues
Session directory	`sess_` files the app would never create; `upload_progress_` keys inside them
HTTP responses	abnormally long / odd-character `Set-Cookie: PHPSESSID=...` - a possible trace of the memory-disclosure vector

Note: a successful **silent** exploitation (pure memory disclosure) may produce no logs at all. So the B group and response-size anomalies should be watched together. The `Set-Cookie` and response-size traces rely on reported vectors; since we did not produce memory disclosure in our lab, treat them as reported (unverified) signals.

07 Traffic and access-log footprint

HTTP status codes alone are insufficient. The observations below come from ORBITRON's controlled tests and should not be treated as universal attack signatures.

- **200 and 404 do not exclude execution:** In the controlled preload/session test, a file-writing request could return 200; startup code also ran when a request to a missing script returned 404. Conversely, a large POST returning 404 is not an attack by itself. Standard access logs do not expose multipart part counts; WAF/body inspection is needed.
- **FPM access-log gap:** Under the examined startup-OOM conditions, the error log and `code 70` were present while the corresponding FPM access entry was absent. Collect front-end nginx/Apache/WAF logs too. Missing entries can also reflect logging configuration, filtering, rotation or collection loss. Absence alone proves neither startup termination, attack nor an unpatched version; do not correlate solely by nearby timestamps.
- **Increasing requests:** Requests to the same target with increasing body/part counts and rising duration may merit investigation for calibration or resource exhaustion. The small lab sample does **not prove O(n²)** complexity. Set thresholds and severity against the application's normal traffic.
- **Volume:** OOM observed with a few hundred KB under a low memory limit cannot be linearly converted to a specific MB threshold at a production 128M limit. Field structure, parser/SAPI, session settings and build differences change the result. Do not use body size alone as a security threshold.
- **Session traces:** Upload progress can update session data without an application call to `session_start()`. `sess_*` files and `upload_progress_` keys also occur during legitimate use; their significance depends on the feature being unexpected in the relevant flow and on supporting context.

Controlled file writing versus the separate OOM test

Test	Prerequisite	Observation	Evidence boundary
File writing: 8.4.26 / 8.4.27	Lab-prepared `opcache.preload` class and seeded session object	Startup `__wakeup` and marker writing on the unpatched build; no marker in the patched control	Execution in a prepared environment, not a full remote memory-corruption chain
Separate multipart/OOM experiment	Requests requiring neither preload nor a seeded session object	Memory consumption, worker exit and log visibility examined	Does not establish UAF or RCE by itself

The file-write test reported **7,634 bytes, 22 appends, UID/GID 65534 and mode 0644** for `hello.txt`. File ownership alone does not identify the writing process; correlate event telemetry with PID, process start time and executable. These names, sizes and UIDs are **lab artefacts, not production IOCs**.

08 Group C - Post-exploitation hunting

Investigate unexpected process, file and network activity from PHP workers. **PHP can write through its own file APIs without starting a new process; monitoring only** `execve` **misses such writes.**

- **auditd (process):** watch `execve` by the web worker user's `euid`:

LOG / PATTERN

```
auditctl -a always,exit -F arch=b64 -S execve -F euid=48 -k php2shell_webexec
```

`/bin/sh`, `/bin/bash`, `curl`, `wget`, `base64`, `perl`, `python` launches (euid = apache is usually 48, www-data 33).

- **Filesystem:** new files in `/tmp`, `/dev/shm`, upload directories and the web root - especially new `.php` (webshell) and oddly named script files.
- **Persistence:** crontab entries, SSH `authorized_keys`, systemd user units owned by the web worker user.
- **Network:** unexpected outbound connections from the PHP worker (reverse shell, C2, data exfiltration).

Collection and attribution: `euid=48` is an example; replace it with the actual pool user's ID. The audit rule covers `execve`, not file writes or `execveat`. Its `php2shell_webexec` key matches the repository's Wazuh process rules. Collect `/var/log/audit/audit.log`; use your distribution's audit configuration for persistence. Add EDR file events, suitable audit rules on relevant directories or FIM for file writes. FIM may show a change without identifying its writer PID. On Windows, collect configured Sysmon Event IDs 1/11 and the product's corresponding file/network telemetry. A new `.txt` marker can evade rules restricted to `.php` extensions.

Compare this activity against change records and the application's normal behaviour; apps that legitimately use `exec()` (image/PDF tools, backups, deploy hooks) produce false positives.

09 Correlation and alerting

PHP error lines may lack a source IP. Obtain client IPs from the trusted proxy chain; correlate using request ID, host/container, pool, worker PID and process start time where available. PIDs can be reused over time. A nearby timestamp or the same pool does not establish that events belong to the same request.

Suggested priority	Condition	Interpretation / action
HIGH	A1 ≥3 / 10 min reliably attributed to one source, with unusual multipart behaviour	Repeated resource exhaustion or possible attempt; prompt triage
HIGH	Unusual multipart + related 5xx/worker respawn + repetition	Service impact/possible attempt; do not assume success
MEDIUM	Unexpected A2/A3 with multipart context	Also investigate session/configuration/storage causes
HIGH; CRITICAL if confirmed	Unauthorised file/command/network activity attributed to a worker	Check change records; initiate IR for confirmed unauthorised activity
MEDIUM	Related A1 + FPM code 70	Consistent with OOM-related startup termination
MEDIUM	Unexplained SIGSEGV/SIGABRT	Investigate the crash; escalate with supporting evidence
Investigation signal	Startup error in error log, no FPM access entry	Validate the logging pipeline first; add front-end records
MEDIUM	≥4 increasing-size requests to one target with increasing duration	Possible calibration; a starting threshold, not proof of attack

These priorities are starting recommendations; confidence and operational severity are different. A critical service outage may require urgent response without confirmed exploitation.

Duplicate events: The same error can arrive through FPM, Apache and the journal. Deduplicate before calculating thresholds such as “3 attempts”.

Record the correct FPM access-log fields

Example pool configuration; adapt the directory, permissions and rotation to your environment:

CONFIG

```
access.log = /var/log/php-fpm/www-access.log
access.format = "%R %t method=%m uri=%r status=%s content_length=%l duration_ms=%{milliseconds}d
  worker_pid=%p pool=%n script=%f"
```

`%p` is the child/worker PID; `%P` is the parent PID. `%l` is the declared POST Content-Length, not the actual received body size. `%{milliseconds}d` explicitly records milliseconds. `%t` includes a time zone; normalise it in the SIEM. Use front-end telemetry for received bytes and check whether its field includes headers. Configure request-ID propagation between proxy and FPM separately. This format cannot recover an FPM access entry that was never emitted during startup.

10 Ready queries and ORBITRON rules

Greps for quick triage:

```shell
# Confirm the actual log paths from configuration.
# A1: PHP/FPM/Apache OOM
grep -E 'Allowed memory size of [0-9]+ bytes exhausted.*in Unknown on line 0' \
  /var/log/php-fpm/*log /var/log/php*-fpm.log \
  /var/log/httpd/*error* /var/log/apache2/*error*
```

```shell
# A2-A4: PHP Request Startup
grep -F 'PHP Request Startup:' \
  /var/log/php-fpm/*log /var/log/php*-fpm.log \
  /var/log/httpd/*error* /var/log/apache2/*error*
```

```shell
# FPM: code 70 / signal
grep -E 'exited with code 70|exited on signal (11|6)' \
  /var/log/php-fpm/*log /var/log/php*-fpm.log
```

```shell
# Apache + FPM
journalctl -u httpd -u apache2 --since today | \
  grep 'AH01071' | grep -E 'Allowed memory|Request Startup'
```

```shell
# nginx combined: POST + 5xx; does not identify multipart.
awk '$6=="\"POST" && $9~/^50[0-9]$/' /var/log/nginx/access.log
```

Mapping to the ORBITRON php2shell-response-kit (public repo):

Signal	Rule
A1 (OOM, in Unknown on line 0)	Sigma `php2shell_php_oom_during_request_parsing` · Wazuh 109627
OOM + then a worker crash (2 min)	Wazuh 109628
Repeated OOM (5/5 min)	Wazuh 109629
FPM worker exited with code 70 (OOM-startup)	Sigma `php2shell_php_fpm_worker_exit_code70` · Wazuh 109630
Code 70 within 2 min of an OOM-during-parsing	Wazuh 109631
Worker crash (SIGSEGV/SIGABRT)	Sigma `php2shell_php_worker_crash` · Wazuh 109625/109626
A6 header flood / body error / strict parsing error	ModSecurity 9109622 / 9109623 / 9109621
Upload-progress field	ModSecurity 9109618 / 9109619
C group (webserver → shell/webshell)	Sigma spawns_shell / file_written_to_webroot · Wazuh 109610-109613, 109615-109616, 109620-109621 · YARA

Test scope: SIEM/EDR queries were generated with pySigma and have not been validated on live platforms. Wazuh XML/regex checks were performed, but live-manager loading and correlation rules were not tested. ModSecurity 9109623 triggering was not exercised; script and YARA coverage is documented in the repository's `tests/TESTED.md`. Native Windows/IIS/PowerShell 5.1 validation of the Windows script is also outstanding. A short baseline helps initial triage; 1-2 hours may not represent all legitimate workloads.

Correlation limitation: Repository rules Wazuh 109628/109631 do not contain field conditions tying the earlier OOM and later event to the same worker PID. Do not report their matches as “OOM confirmed on this worker”. Validate host/agent scoping on the live manager; exact process attribution requires suitable decoding and PID/pool correlation. This guide's priority recommendations do not automatically change the existing rule levels.

The grep examples do not cover rotated/compressed logs. Nonexistent paths produce errors; adapt the actual source list. The `awk` example assumes standard combined-log field order and does not inspect multipart bodies.

11 Interim mitigations, patch verification and sources

Priority: Apply the fixed upstream version or a verified vendor backport addressing both CVEs. Restart the relevant services and long-lived workers; verify the processes serving web requests, not just the CLI version on disk. For containers, redeploy the patched image and verify that old pods/containers have terminated.

The following are **partial interim measures, not a substitute for patching**. Apply them to the effective web-SAPI configuration. Example FPM pool settings:

```
CONFIG
php_admin_value[memory_limit] = -1
php_admin_value[post_max_size] = 8M
php_admin_flag[session.upload_progress.enabled] = off
; PHP 8.5 only:
php_admin_flag[fatal_error_backtraces] = off
```

For applications requiring uploads, assess PHP's recommended 32M `post_max_size` against the workload; `upload_max_filesize` should be lower. Changing a setting later with `ini_set()` is too late for startup paths. Check that other pool/directory settings do not restore a finite memory limit. Test configuration before activation, replace all workers and verify the effective web-SAPI settings.

Measure	Monitoring effect and limitation
`session.upload_progress.enabled=Off`	Disables the relevant early session path; other causes of generic messages and other CVE paths remain
`memory_limit=-1` + resource limits	Reduces A1 caused by the PHP limit; allocation failures and DoS remain possible. Monitor kernel/cgroup OOM and restarts
Low `post_max_size` + proxy/body/part/header limits	Rejects covered requests, not every attack path. Also constrain data without usable Content-Length
Routing only to required PHP scripts	May reduce unnecessary paths rejected before PHP; verify front-controller/rewrite behaviour
`fatal_error_backtraces=Off` (8.5)	Removes one confirmed stale-state producer, not all state-lifecycle paths
Fix invalid/deprecated early INI settings	Reduces one early diagnostic consumer path; hiding warnings alone provides no protection

Visibility: Collect PHP errors, FPM process/access records, web/WAF records and endpoint file/process/network events together. `display_errors=Off` does not disable logging; enable `log_errors` and relevant error levels. `catch_workers_output` helps forward worker stdout/stderr but does not replace collection of PHP errors written to a separate file. Preserve existing evidence; if unauthorised activity is confirmed, isolate and investigate scope through the IR process.

Sources:

- PHP security releases 8.5.12 / 8.4.27 / 8.3.36 / 8.2.35 (21 Oct 2026): php.net/archive/2026
- Fix commit "Avoid state corruption via bailout" (GHSA-w8hw-w9fx-j79w, GHSA-6wmf-4rv5-p3rp): github.com/php/php-src
- ORBITRON php2shell-response-kit (inventory scripts + rules): github.com/orbitron-ai/php2shell-response-kit
- Official CVE advisories: OOM corruption, stale global state
- FPM log fields: PHP FPM configuration
- Rule test coverage: TESTED.md

The lab verification was done by ORBITRON on 21 Oct 2026 in an isolated environment, using unpatched/patched builds compiled from PHP's official source and PHP's official regression tests. No exploit chain was built and nothing was triggered on production systems.

12 Document control

Document owner: **ORBITRON Team** · Document ID: **ORB-PHP2SHELL-EN**

Version	Date	Change
1.0	21.10.2026	Initial branded layout and document control
1.1	21.10.2026	Controlled file writing, OOM separation and telemetry additions
1.2	21.10.2026	Reconciled with final GitHub texts; corrected evidence boundaries, FPM scope, rule mappings and collection examples